titanic_classification_models

September 24, 2026

1 Classification Algorithms on the Titanic Dataset

1.0.1 Logistic Regression | Decision Trees & Random Forests | Support Vector Machines (SVM) | K-Nearest Neighbours (KNN)

Module: Classification Algorithms in Machine Learning **Dataset:** Titanic — Machine Learning from Disaster (`titanic.csv`) **Prepared for:** Python for Data Science Course

---

This notebook walks through four core classification algorithms end-to-end on the same dataset, so their preprocessing, training workflow, evaluation metrics, and results can be directly compared side by side.

Contents 1. Setup & Imports 2. Load the Data 3. Exploratory Data Analysis (EDA) 4. Data Cleaning & Preprocessing 5. Train/Test Split 6. Logistic Regression 7. Decision Tree Classifier 8. Random Forest Classifier 9. Support Vector Machine (SVM) 10. K-Nearest Neighbours (KNN) 11. Model Comparison 12. Summary & Exercises

1.1 1. Setup & Imports

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report,
                             roc_curve, roc_auc_score)

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (7, 5)
RANDOM_STATE = 42
```

```
pd.set_option("display.max_columns", None)
```

1.2 2. Load the Data

The classic Kaggle Titanic dataset: 891 passengers with demographic and travel details, and a binary target `Survived` (1 = survived, 0 = did not survive).

[2]:
```
df = pd.read_csv("titanic.csv")
print("Shape:", df.shape)
df.head()
```

```
Shape: (891, 12)
```

[2]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th…	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

[3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
```

```
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

[4]:
```
df.describe(include="all").T
```

[4]:

	count	unique	top	freq	mean	\
PassengerId	891.0	NaN	NaN	NaN	446.0	
Survived	891.0	NaN	NaN	NaN	0.383838	
Pclass	891.0	NaN	NaN	NaN	2.308642	
Name	891	891	Braund, Mr. Owen Harris	1	NaN	
Sex	891	2	male	577	NaN	
Age	714.0	NaN	NaN	NaN	29.699118	
SibSp	891.0	NaN	NaN	NaN	0.523008	
Parch	891.0	NaN	NaN	NaN	0.381594	
Ticket	891	681	347082	7	NaN	
Fare	891.0	NaN	NaN	NaN	32.204208	
Cabin	204	147	G6	4	NaN	
Embarked	889	3	S	644	NaN	

	std	min	25%	50%	75%	max
PassengerId	257.353842	1.0	223.5	446.0	668.5	891.0
Survived	0.486592	0.0	0.0	0.0	1.0	1.0
Pclass	0.836071	1.0	2.0	3.0	3.0	3.0
Name	NaN	NaN	NaN	NaN	NaN	NaN
Sex	NaN	NaN	NaN	NaN	NaN	NaN
Age	14.526497	0.42	20.125	28.0	38.0	80.0
SibSp	1.102743	0.0	0.0	0.0	1.0	8.0
Parch	0.806057	0.0	0.0	0.0	0.0	6.0
Ticket	NaN	NaN	NaN	NaN	NaN	NaN
Fare	49.693429	0.0	7.9104	14.4542	31.0	512.3292
Cabin	NaN	NaN	NaN	NaN	NaN	NaN
Embarked	NaN	NaN	NaN	NaN	NaN	NaN

1.3 3. Exploratory Data Analysis (EDA)

[5]:
```
print("Missing values per column:")
df.isnull().sum().sort_values(ascending=False)
```

```
Missing values per column:
```

[5]:
```
Cabin          687
Age            177
Embarked         2
PassengerId      0
```

```
Name            0
Pclass          0
Survived        0
Sex             0
Parch           0
SibSp           0
Fare            0
Ticket          0
dtype: int64
```

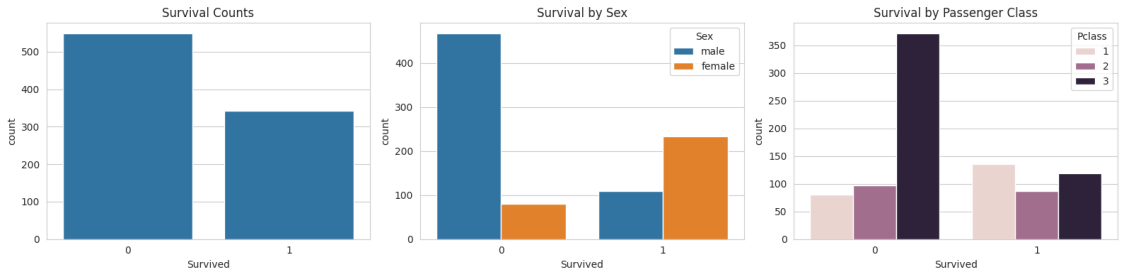
[6]:
```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

sns.countplot(x="Survived", data=df, ax=axes[0])
axes[0].set_title("Survival Counts")

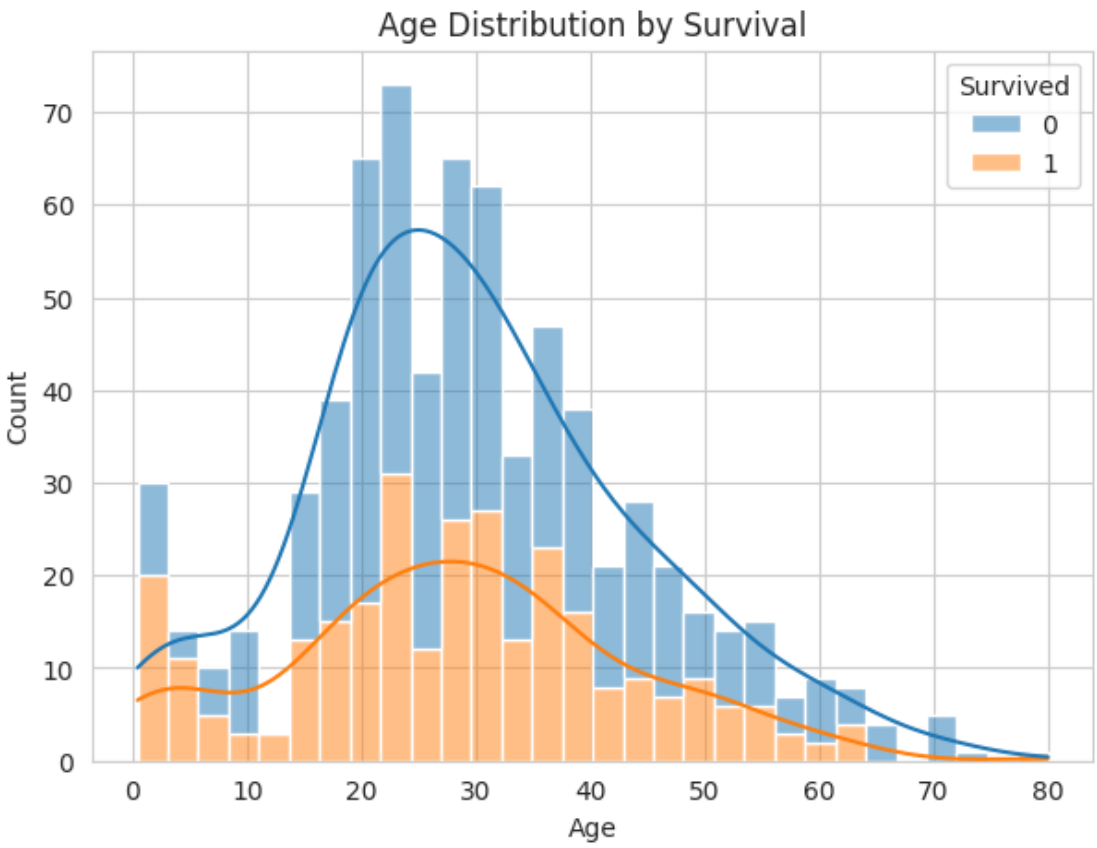
sns.countplot(x="Survived", hue="Sex", data=df, ax=axes[1])
axes[1].set_title("Survival by Sex")

sns.countplot(x="Survived", hue="Pclass", data=df, ax=axes[2])
axes[2].set_title("Survival by Passenger Class")

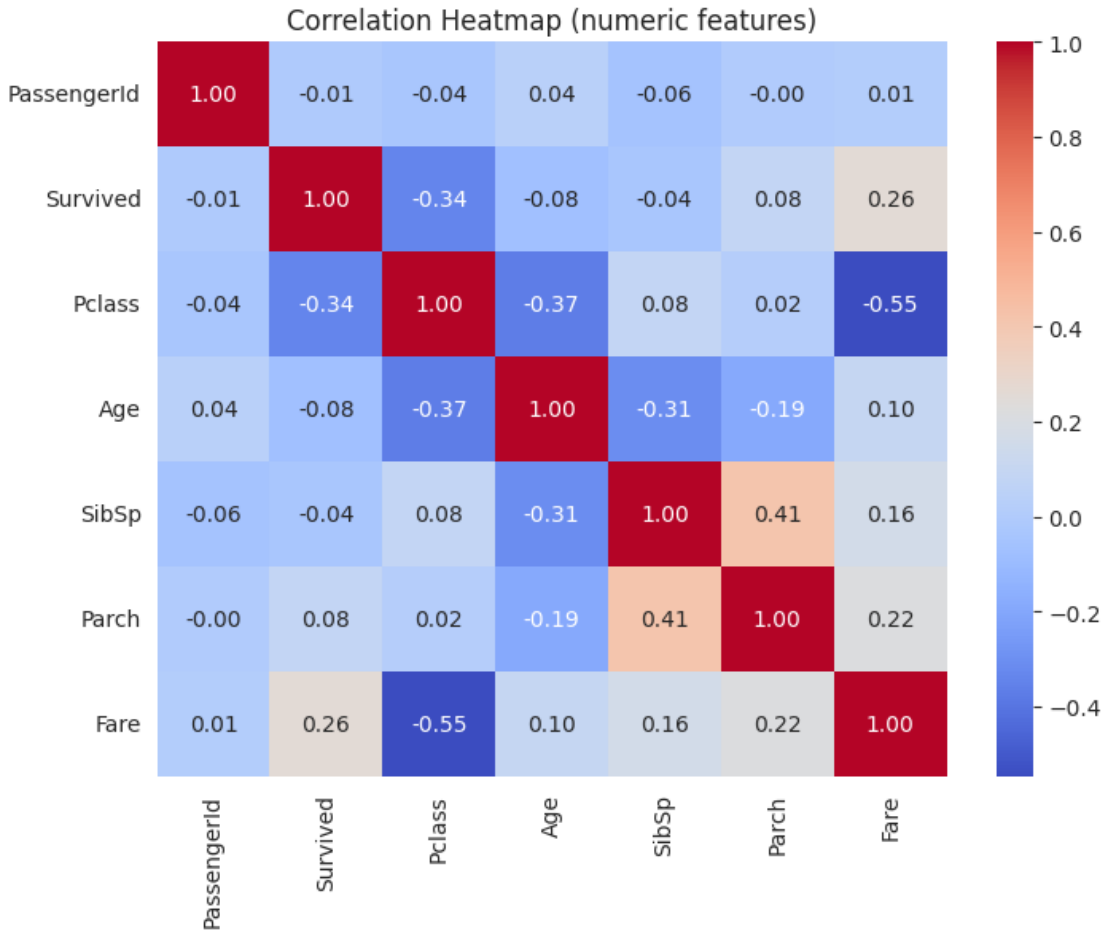
plt.tight_layout()
plt.show()
```



[7]:
```
plt.figure(figsize=(7, 5))
sns.histplot(data=df, x="Age", hue="Survived", multiple="stack", bins=30,␣
↪kde=True)
plt.title("Age Distribution by Survival")
plt.show()
```



[8]:
```
numeric_df = df.select_dtypes(include=[np.number])
plt.figure(figsize=(8, 6))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap (numeric features)")
plt.show()
```



1.4 4. Data Cleaning & Preprocessing

Steps applied: - Drop high-cardinality / mostly-missing / non-predictive columns: `PassengerId`, `Name`, `Ticket`, `Cabin` - Impute missing `Age` with the median, missing `Embarked` with the mode - Encode `Sex` and `Embarked` as numeric via one-hot / label encoding - Engineer `FamilySize` = `SibSp` + `Parch` + 1 - Scale numeric features for the distance/margin-based models (KNN, SVM)

[9]:
```
data = df.drop(columns=["PassengerId", "Name", "Ticket", "Cabin"])

data["Age"] = data["Age"].fillna(data["Age"].median())
data["Embarked"] = data["Embarked"].fillna(data["Embarked"].mode()[0])

data["FamilySize"] = data["SibSp"] + data["Parch"] + 1

data["Sex"] = data["Sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(data, columns=["Embarked"], drop_first=True)

print("Missing values after cleaning:", data.isnull().sum().sum())
```

```
data.head()
```

```
Missing values after cleaning: 0
```

[9]:

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	FamilySize	Embarked_Q	Embarked_S
0	0	3	0	22.0	1	0	7.2500	2	False	True
1	1	1	1	38.0	1	0	71.2833	2	False	False
2	1	3	1	26.0	0	0	7.9250	1	False	True
3	1	1	1	35.0	1	0	53.1000	2	False	True
4	0	3	0	35.0	0	0	8.0500	1	False	True

1.5 5. Train/Test Split

[10]:
```
X = data.drop(columns=["Survived"])
y = data["Survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Train shape:", X_train.shape, " Test shape:", X_test.shape)
```

```
Train shape: (712, 9)  Test shape: (179, 9)
```

[11]:
```
results = {}

def evaluate_model(name, y_true, y_pred, y_proba=None):
    """Compute and store standard classification metrics for a model."""
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred)
    rec = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    auc = roc_auc_score(y_true, y_proba) if y_proba is not None else np.nan

    results[name] = {"Accuracy": acc, "Precision": prec, "Recall": rec,
                     "F1-score": f1, "ROC-AUC": auc}
```

```
    print(f"--- {name} ---")
    print(classification_report(y_true, y_pred, target_names=["Died",␣
↪"Survived"]))

    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(4, 3.5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=["Died", "Survived"], yticklabels=["Died",␣
↪"Survived"])
    plt.title(f"{name} - Confusion Matrix")
    plt.ylabel("Actual")
    plt.xlabel("Predicted")
    plt.show()
```

1.6 6. Logistic Regression

A linear model that estimates the probability of survival via the logistic (sigmoid) function. It is fast, interpretable, and a strong baseline for binary classification.

[12]:
```
log_reg = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
log_reg.fit(X_train_scaled, y_train)

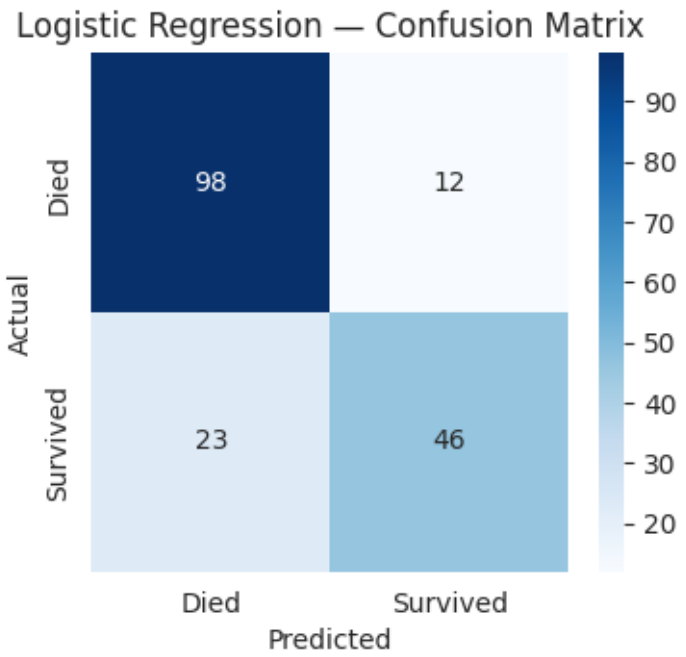
y_pred_lr = log_reg.predict(X_test_scaled)
y_proba_lr = log_reg.predict_proba(X_test_scaled)[:, 1]

evaluate_model("Logistic Regression", y_test, y_pred_lr, y_proba_lr)
```

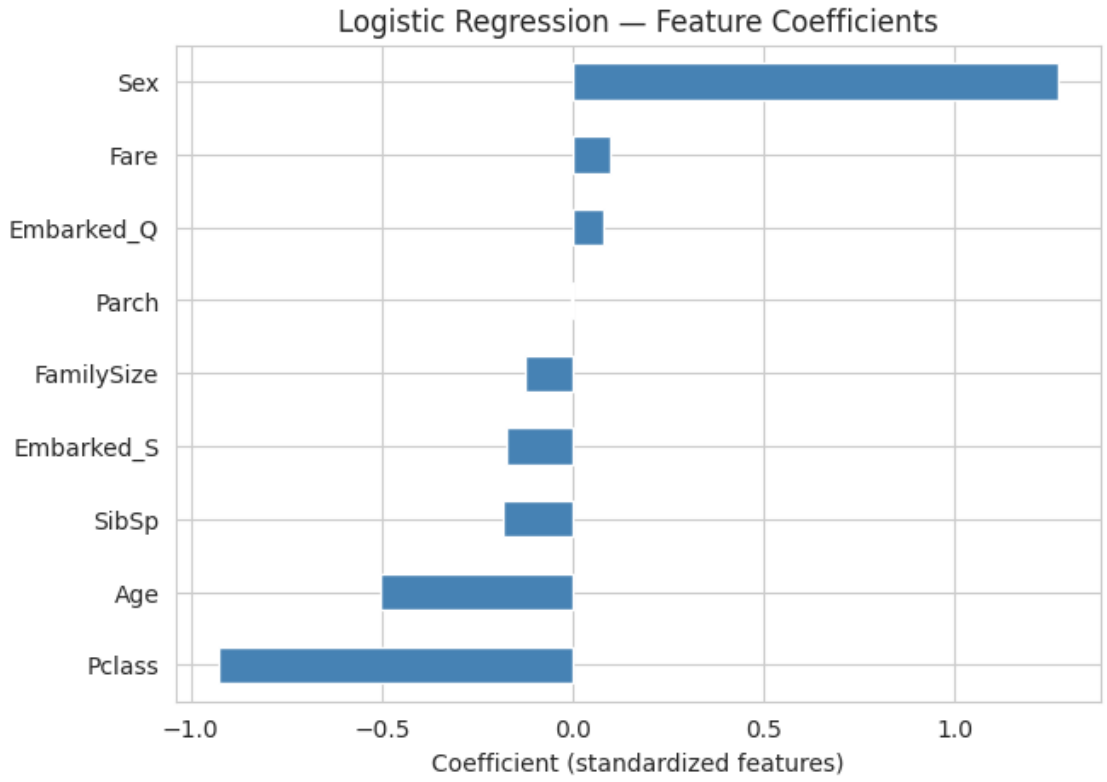
```
--- Logistic Regression ---
              precision    recall  f1-score   support

        Died       0.81      0.89      0.85       110
    Survived       0.79      0.67      0.72        69

    accuracy                           0.80       179
   macro avg       0.80      0.78      0.79       179
weighted avg       0.80      0.80      0.80       179
```



[13]:
```
coefs = pd.Series(log_reg.coef_[0], index=X.columns).sort_values()
plt.figure(figsize=(7, 5))
coefs.plot(kind="barh", color="steelblue")
plt.title("Logistic Regression - Feature Coefficients")
plt.xlabel("Coefficient (standardized features)")
plt.show()
```



1.7 7. Decision Tree Classifier

A rule-based model that recursively splits the data on the feature/threshold that most reduces impurity (Gini index by default). Easy to visualize and interpret, but prone to overfitting if left unconstrained — here `max_depth` is limited.

[14]:
```
dt = DecisionTreeClassifier(max_depth=4, random_state=RANDOM_STATE)
dt.fit(X_train, y_train)

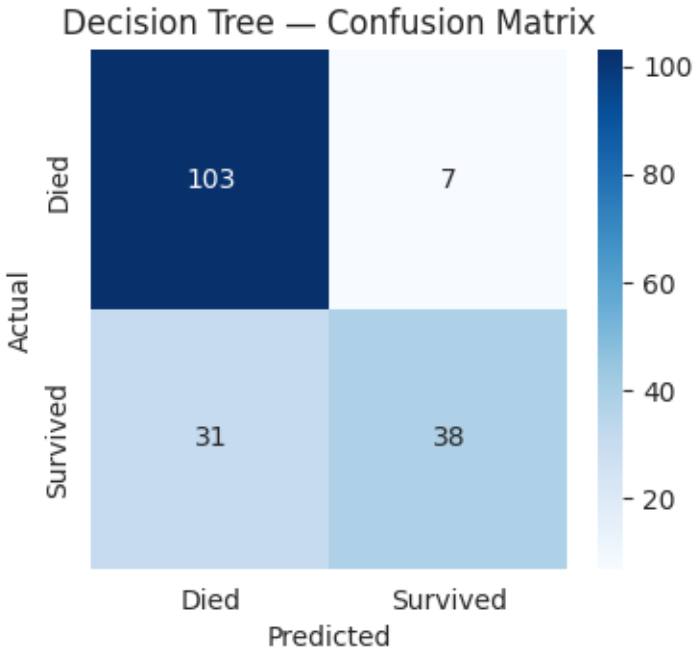
y_pred_dt = dt.predict(X_test)
y_proba_dt = dt.predict_proba(X_test)[:, 1]

evaluate_model("Decision Tree", y_test, y_pred_dt, y_proba_dt)
```

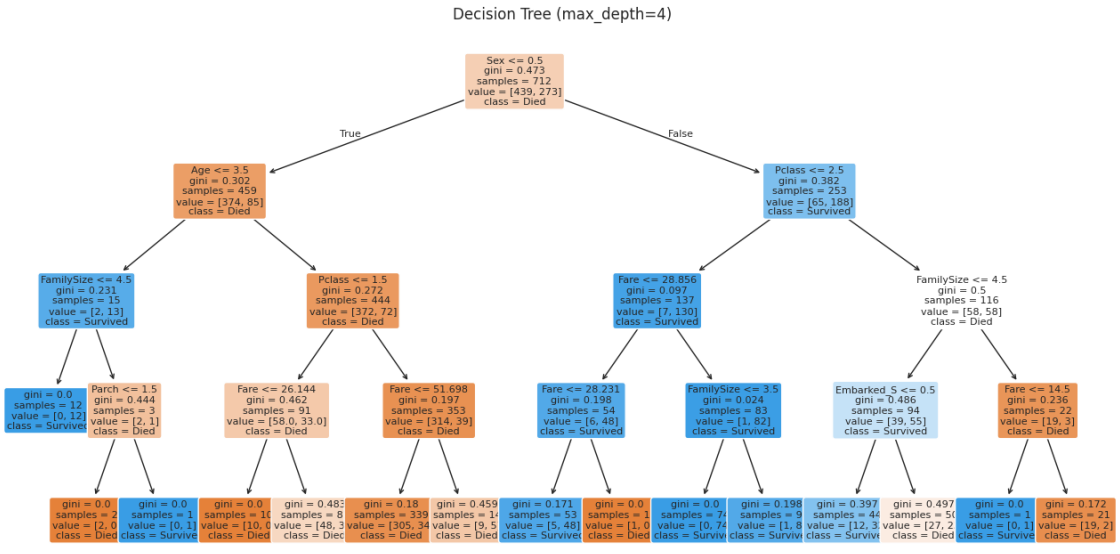
```
--- Decision Tree ---
              precision    recall  f1-score   support

        Died       0.77      0.94      0.84       110
    Survived       0.84      0.55      0.67        69

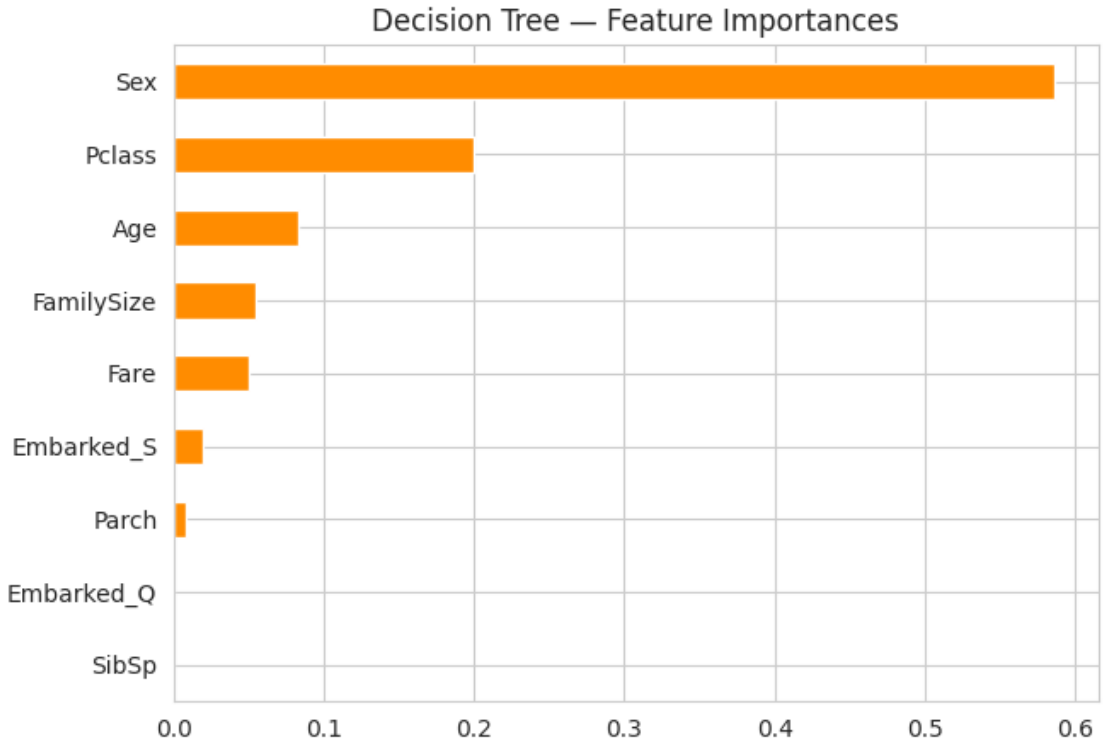
    accuracy                           0.79       179
   macro avg       0.81      0.74      0.76       179
weighted avg       0.80      0.79      0.78       179
```



[15]:
```
plt.figure(figsize=(16, 8))
plot_tree(dt, feature_names=X.columns, class_names=["Died", "Survived"],
          filled=True, rounded=True, fontsize=8)
plt.title("Decision Tree (max_depth=4)")
plt.show()
```



[16]:
```
importances = pd.Series(dt.feature_importances_, index=X.columns).sort_values()
plt.figure(figsize=(7, 5))
importances.plot(kind="barh", color="darkorange")
plt.title("Decision Tree - Feature Importances")
plt.show()
```



1.8 8. Random Forest Classifier

An ensemble of many decision trees, each trained on a bootstrap sample with a random subset of features; predictions are combined by majority vote. This reduces the overfitting/variance problem of a single tree.

[17]:
```
rf = RandomForestClassifier(n_estimators=300, max_depth=6,␣
  ↪random_state=RANDOM_STATE)
rf.fit(X_train, y_train)

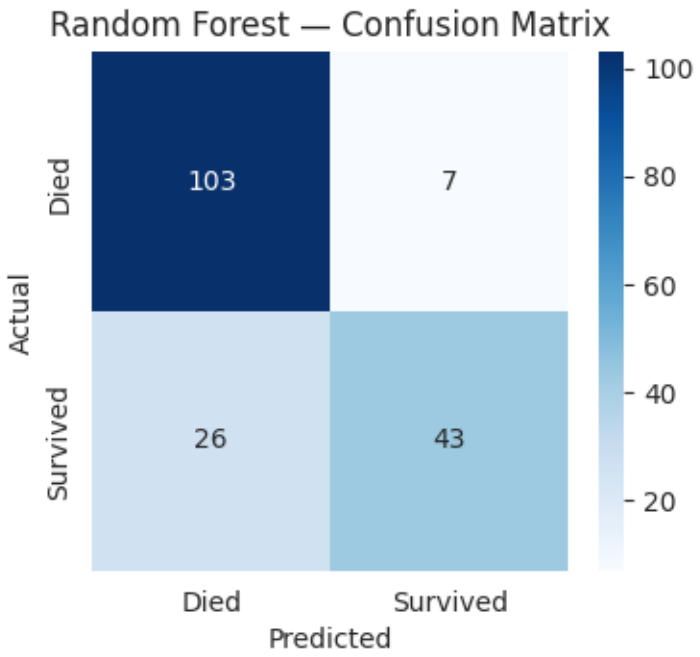
y_pred_rf = rf.predict(X_test)
y_proba_rf = rf.predict_proba(X_test)[:, 1]

evaluate_model("Random Forest", y_test, y_pred_rf, y_proba_rf)
```

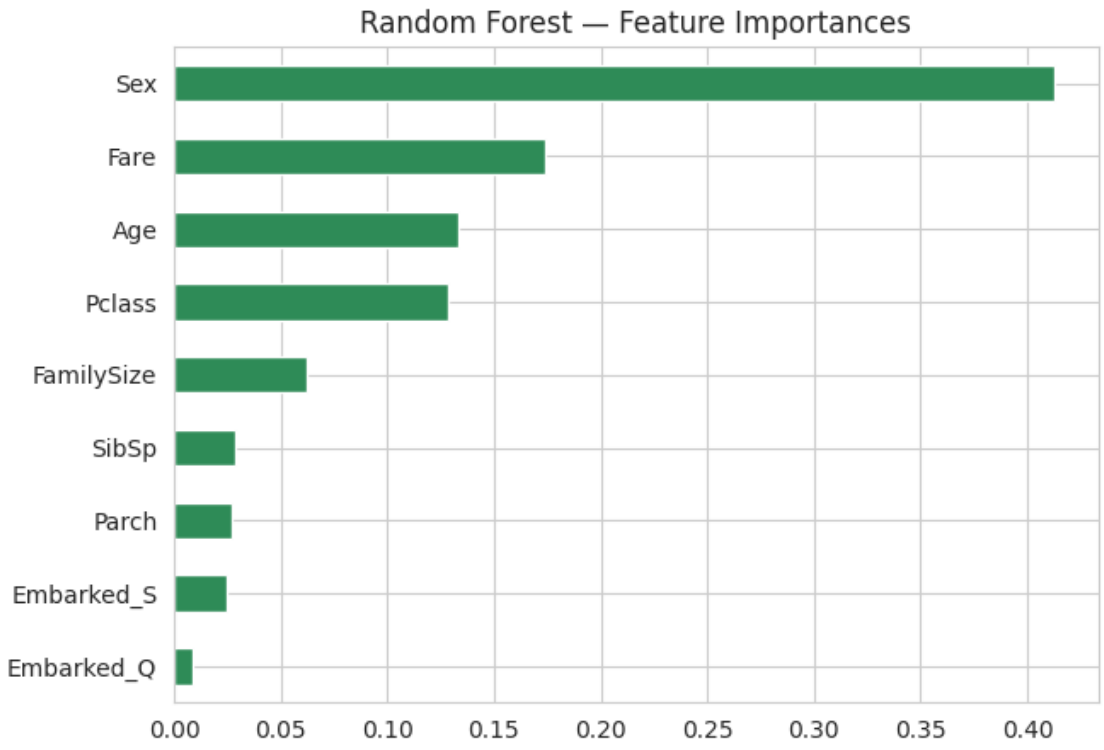
```
--- Random Forest ---
              precision    recall  f1-score   support

        Died       0.80      0.94      0.86       110
    Survived       0.86      0.62      0.72        69

    accuracy                           0.82       179
   macro avg       0.83      0.78      0.79       179
weighted avg       0.82      0.82      0.81       179
```



[18]:
```
importances_rf = pd.Series(rf.feature_importances_, index=X.columns).
 ↪sort_values()
plt.figure(figsize=(7, 5))
importances_rf.plot(kind="barh", color="seagreen")
plt.title("Random Forest - Feature Importances")
plt.show()
```



1.9 9. Support Vector Machine (SVM)

Finds the hyperplane that maximizes the margin between classes. Here we use an RBF kernel to capture non-linear decision boundaries. SVM is sensitive to feature scale, so we use the standardized features.

[19]:
```
svm = SVC(kernel="rbf", C=1.0, gamma="scale", probability=True,␣
 ↪random_state=RANDOM_STATE)
svm.fit(X_train_scaled, y_train)

y_pred_svm = svm.predict(X_test_scaled)
y_proba_svm = svm.predict_proba(X_test_scaled)[:, 1]

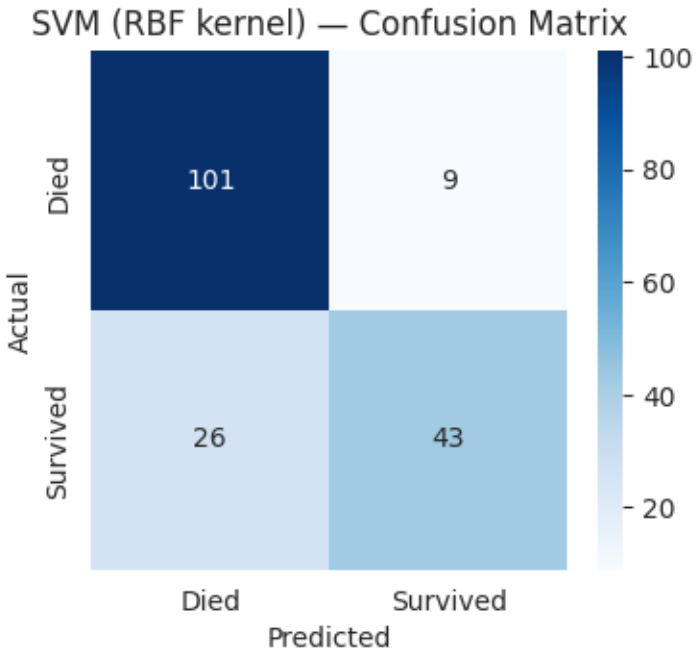
evaluate_model("SVM (RBF kernel)", y_test, y_pred_svm, y_proba_svm)
```

```
--- SVM (RBF kernel) ---
              precision    recall  f1-score   support

        Died       0.80      0.92      0.85       110
    Survived       0.83      0.62      0.71        69

    accuracy                           0.80       179
   macro avg       0.81      0.77      0.78       179
```

```
weighted avg       0.81      0.80      0.80       179
```



Optional: quick hyperparameter search for `C` and `gamma`

[20]:
```
param_grid = {"C": [0.1, 1, 10], "gamma": ["scale", 0.01, 0.1]}
grid_svm = GridSearchCV(SVC(kernel="rbf", probability=True,␣
 ↪random_state=RANDOM_STATE),
                        param_grid, cv=5, scoring="accuracy")
grid_svm.fit(X_train_scaled, y_train)

print("Best params:", grid_svm.best_params_)
print("Best CV accuracy: %.4f" % grid_svm.best_score_)
```

```
Best params: {'C': 10, 'gamma': 0.1}
Best CV accuracy: 0.8273
```

1.10 10. K-Nearest Neighbours (KNN)

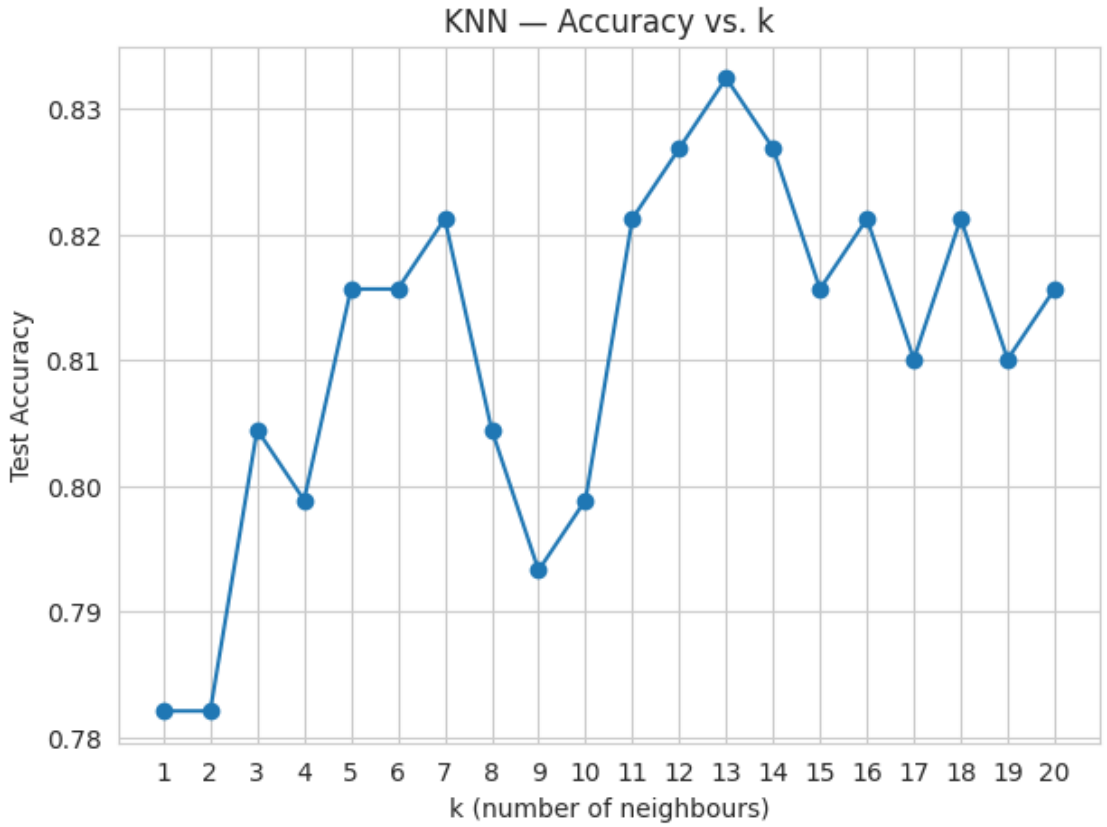
Classifies a point by majority vote among its `k` nearest neighbours in feature space. Like SVM, it is distance-based, so scaled features are essential. We first sweep over `k` to find a good value, then fit the final model.

[21]:
```
k_values = range(1, 21)
accuracies = []
```

```
for k in k_values:
    knn_k = KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(X_train_scaled, y_train)
    accuracies.append(accuracy_score(y_test, knn_k.predict(X_test_scaled)))

plt.figure(figsize=(7, 5))
plt.plot(list(k_values), accuracies, marker="o")
plt.title("KNN - Accuracy vs. k")
plt.xlabel("k (number of neighbours)")
plt.ylabel("Test Accuracy")
plt.xticks(list(k_values))
plt.show()

best_k = list(k_values)[int(np.argmax(accuracies))]
print("Best k:", best_k, " Accuracy: %.4f" % max(accuracies))
```




```
Best k: 13  Accuracy: 0.8324
```

[22]:
```
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train_scaled, y_train)
```

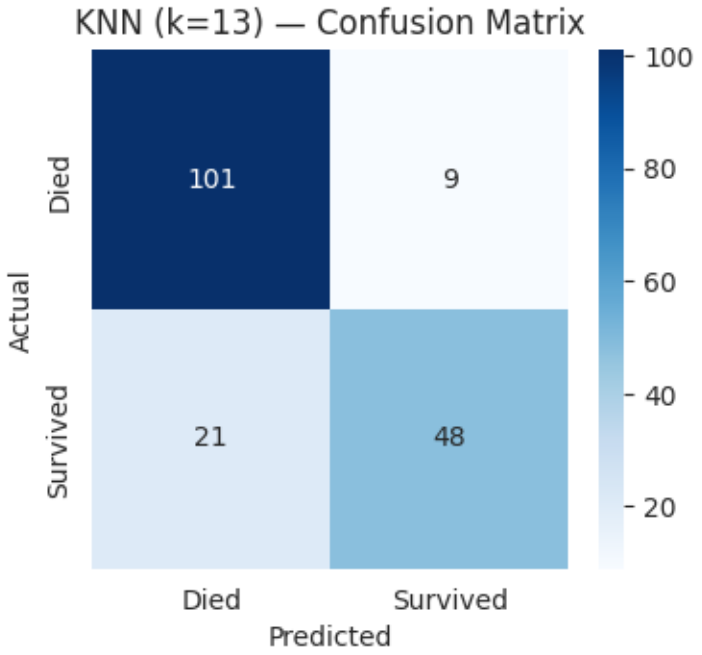
```
y_pred_knn = knn.predict(X_test_scaled)
y_proba_knn = knn.predict_proba(X_test_scaled)[:, 1]

evaluate_model(f"KNN (k={best_k})", y_test, y_pred_knn, y_proba_knn)
```

```
--- KNN (k=13) ---
              precision    recall  f1-score   support

        Died       0.83      0.92      0.87       110
    Survived       0.84      0.70      0.76        69

    accuracy                           0.83       179
   macro avg       0.83      0.81      0.82       179
weighted avg       0.83      0.83      0.83       179
```



1.11 11. Model Comparison

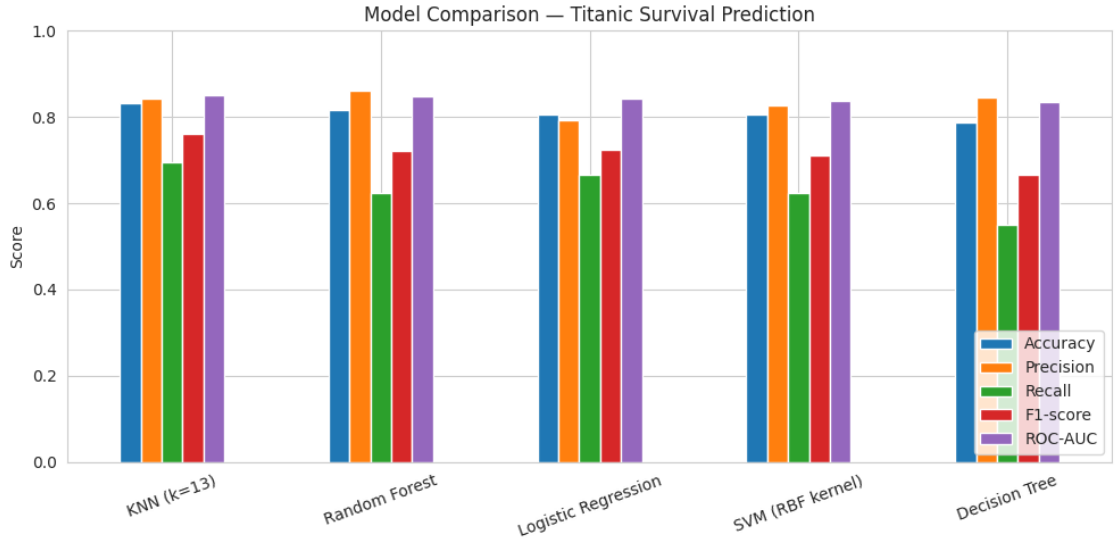
[23]:
```
results_df = pd.DataFrame(results).T.sort_values("Accuracy", ascending=False)
results_df.round(4)
```

[23]:

	Accuracy	Precision	Recall	F1-score	ROC-AUC
KNN (k=13)	0.8324	0.8421	0.6957	0.7619	0.8493
Random Forest	0.8156	0.8600	0.6232	0.7227	0.8481
Logistic Regression	0.8045	0.7931	0.6667	0.7244	0.8436

```
SVM (RBF kernel)        0.8045     0.8269  0.6232    0.7107   0.8372
Decision Tree           0.7877     0.8444  0.5507    0.6667   0.8342
```

[24]:
```
results_df[["Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC"]].plot(
    kind="bar", figsize=(10, 5))
plt.title("Model Comparison - Titanic Survival Prediction")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=20)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

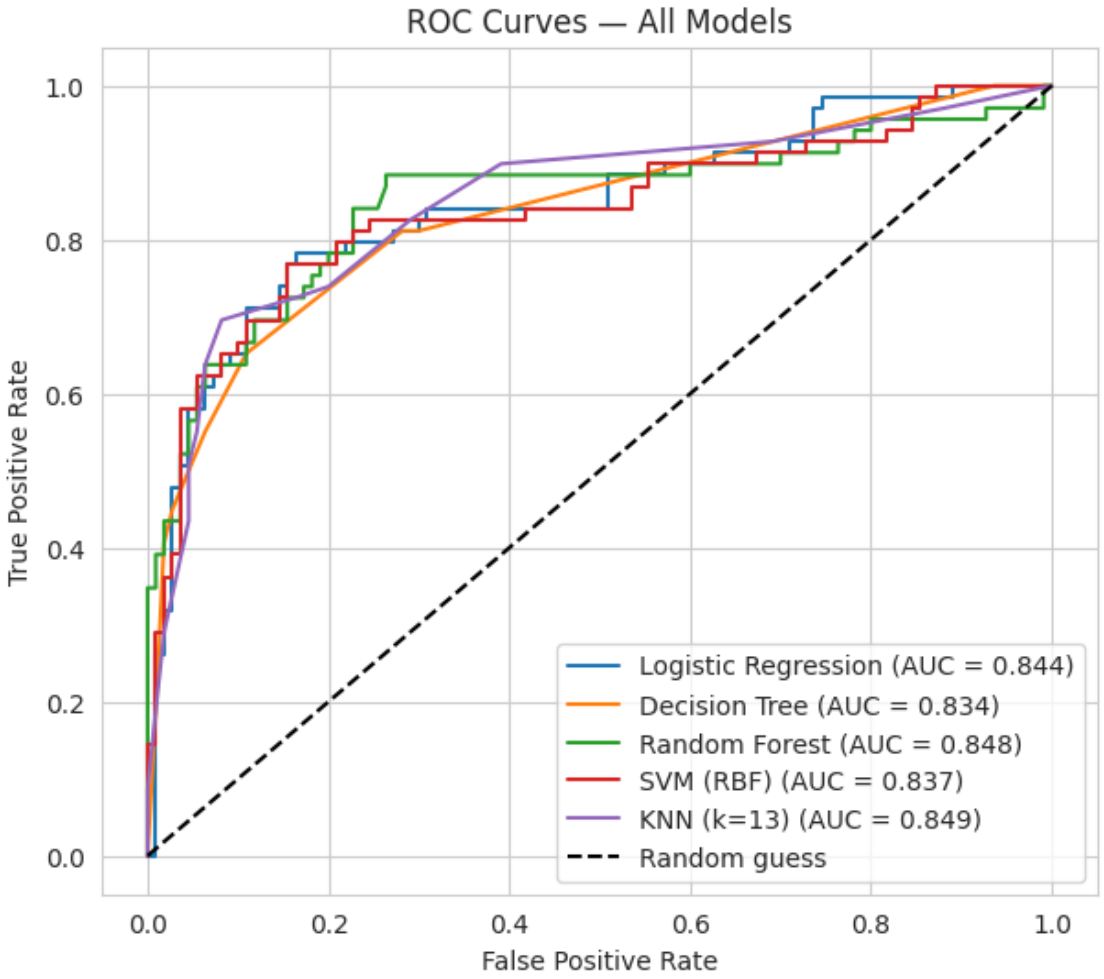


[25]:
```
plt.figure(figsize=(7, 6))

for name, y_proba in [("Logistic Regression", y_proba_lr),
                      ("Decision Tree", y_proba_dt),
                      ("Random Forest", y_proba_rf),
                      ("SVM (RBF)", y_proba_svm),
                      (f"KNN (k={best_k})", y_proba_knn)]:
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    auc = roc_auc_score(y_test, y_proba)
    plt.plot(fpr, tpr, label=f"{name} (AUC = {auc:.3f})")

plt.plot([0, 1], [0, 1], "k--", label="Random guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves - All Models")
```

```
plt.legend(loc="lower right")
plt.show()
```



1.12 12. Summary & Exercises

Key takeaways - **Logistic Regression** — simple, fast, and interpretable; a good baseline. - **Decision Tree** — highly interpretable (visualize the rules), but a single tree can overfit; depth was capped here. - **Random Forest** — usually the strongest of the four on tabular data like this; trades some interpretability for accuracy and stability. - **SVM (RBF)** — can capture non-linear boundaries; needs scaled features and careful tuning of `C` / `gamma`. - **KNN** — simple and non-parametric; sensitive to feature scaling and the choice of `k`; can be slower to predict on large datasets.

Exercises for students 1. Re-run the pipeline using `Title` extracted from `Name` (Mr/Mrs/Miss/Master/…) as an additional feature. Does accuracy improve? 2. Tune `max_depth` and `min_samples_leaf` for the Decision Tree and Random Forest using `GridSearchCV`. Compare against the defaults used here. 3. Try a linear-kernel SVM instead of RBF and compare the ROC-

AUC. 4. Use `cross_val_score` (5-fold) for each model instead of a single train/test split, and report mean ± standard deviation of accuracy. 5. Which model would you deploy, and why — consider both accuracy and interpretability.